Probability distributions and the Gaussian

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Every method in this course either estimates a probability distribution or makes predictions that are best understood through one. This note collects the probability we need — how distributions are described, the four things we do with them, and the multivariate Gaussian in detail — and connects it to two pillars of modern machine learning: maximum-likelihood training of language models and Gaussian noise in diffusion models. Every formula is worked through with small numbers, the derivations are shown step by step, and each algorithm is given as pseudocode with its running time. Notation follows Note 1, Section 2.

1 Describing a distribution

A random variable X is described by how its probability is spread over its possible values. Three descriptions are in common use, plus a fourth, empirical one built from data.

Cumulative distribution function. For a real-valued X (integer-valued ones included), the c.d.f. is $F(x) = P(X \le x)$. It is non-decreasing, tends to 0 as $x \to -\infty$ and to 1 as $x \to +\infty$, and it exists for every random variable, discrete or continuous. Probabilities of intervals follow from it, $P(a < X \le b) = F(b) - F(a)$. For a vector $\mathbf{X} = (X_1, \dots, X_d)$ the inequality is read component-wise: $F(\mathbf{x}) = P(X_1 \le x_1, \dots, X_d \le x_d)$. The definition needs ordered values. For unordered categories, such as particle types or text tokens, one first fixes an arbitrary order; Section 2.2 shows that sampling with the resulting c.d.f. is correct whatever order is chosen.

Probability table (discrete variables). If X takes finitely or countably many values, its distribution is the list of probabilities $P(X = x)$, which are non-negative and sum to one. For several discrete variables the table becomes an array with one axis per variable; a joint table for d variables with m values each has m^d entries — a first sign of the difficulty Note 3 studies.

Density (continuous variables). If F is differentiable, the probability density function is $p(x) = F'(x)$; in d dimensions $p(\mathbf{x}) = \partial^d F / \partial x_1 \cdots \partial x_d$. Probabilities of regions are integrals,

$$P(\mathbf{X} \in A) = \int_A p(\mathbf{x})\, d\mathbf{x}, \qquad \int_{\mathbb{R}^d} p(\mathbf{x})\, d\mathbf{x} = 1.$$

A density is not a probability: it carries units of $1/[x]$ and may exceed one. Only $p(x)\,\Delta x$, for a small interval Δx, is a probability. A Gaussian of width $\sigma = 0.1$ mm has a peak density of about 4 mm^{-1}, yet the probability of landing in any interval of width 0.01 mm is at most 0.04.

The empirical distribution. Given a sample $x_1, \dots, x_n$, the empirical c.d.f. $\hat{F}(x) = \frac{1}{n}\#\{i : x_i \le x\}$ is a step function that jumps by $1/n$ at each data point (by k/n at a value that occurs k times). For the five measurements $4, 7, 5, 9, 5$ of Section 6.1, $\hat{F}(3.9) = 0$, $\hat{F}(4) = 1/5$, $\hat{F}(5) = \hat{F}(6.5) = 3/5$ (a jump of $2/5$ at the repeated value 5) and $\hat{F}(9) = 1$. By the law of large numbers $\hat{F}(x) \to F(x)$ for every x; the convergence is even uniform in x (Glivenko–Cantelli). The empirical distribution is the simplest estimate of a distribution, and most of the estimators in this course can be viewed as ways of smoothing it. To evaluate $\hat{F}$ at many points, sort the sample once ($O(n \log n)$) and locate each query by binary search ($O(\log n)$ per query, $O(n)$ memory), the same search as in Section 2.2.

1.1 Joint, marginal and conditional

For two variables with joint density $p(\mathbf{x}, \mathbf{z})$ (sums replace integrals in the discrete case):

$$\begin{aligned} \text{marginal:} \quad & p(\mathbf{x}) = \int p(\mathbf{x}, \mathbf{z})\, d\mathbf{z}, \\ \text{conditional:} \quad & p(\mathbf{z} \mid \mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})} \qquad (p(\mathbf{x}) > 0), \\ \text{Bayes' rule:} \quad & p(\mathbf{z} \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})}{\int p(\mathbf{x} \mid \mathbf{z}')\, p(\mathbf{z}')\, d\mathbf{z}'}. \end{aligned}$$

Bayes' rule is not a new axiom. The definition of the conditional writes the joint in two ways, the *product rule* $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z} \mid \mathbf{x})\, p(\mathbf{x}) = p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})$. Dividing both sides by $p(\mathbf{x})$ gives $p(\mathbf{z} \mid \mathbf{x}) = p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z}) / p(\mathbf{x})$, and writing $p(\mathbf{x})$ as the marginal of the joint, $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}')\, p(\mathbf{z}')\, d\mathbf{z}'$, gives the denominator. Independence means $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x})\, p(\mathbf{z})$, equivalently $p(\mathbf{z} \mid \mathbf{x}) = p(\mathbf{z})$.

Worked example — a joint table of two detector flags

Two trigger flags fire or not on each recorded event: $A = 1$ if the calorimeter fires, $B = 1$ if the muon chambers fire. Suppose their joint table is the one on the right, with the marginals in the margins.

	$B = 0$	$B = 1$	$P(A)$
$A = 0$	0.50	0.10	0.60
$A = 1$	0.15	0.25	0.40
$P(B)$	0.65	0.35	1

Marginals are row and column sums: $P(A = 1) = 0.15 + 0.25 = 0.40$ and $P(B = 1) = 0.10 + 0.25 = 0.35$. *Conditionals* renormalise one row: $P(B = 1 \mid A = 1) = 0.25/0.40 = 0.625$, while $P(B = 1 \mid A = 0) = 0.10/0.60 \approx 0.167$. Knowing that the calorimeter fired raises the probability of a muon-chamber signal from 0.35 to 0.625, so the flags are *not* independent; equivalently, $P(A = 1)\, P(B = 1) = 0.14 \neq 0.25 = P(A = 1, B = 1)$.

Worked example — Bayes' rule for particle identification

In a sample of charged tracks, 10% are kaons and 90% pions. A particle-identification algorithm tags a true kaon with probability 0.90 (its efficiency) and wrongly tags a pion with probability 0.05 (its mis-identification rate). What fraction of tagged tracks are kaons? With K = "is a kaon" and T = "is tagged",

$$P(K \mid T) = \frac{P(T \mid K)\, P(K)}{P(T \mid K)\, P(K) + P(T \mid \bar{K})\, P(\bar{K})} = \frac{0.90 \times 0.10}{0.90 \times 0.10 + 0.05 \times 0.90} = \frac{0.090}{0.135} \approx 0.67.$$

Although the tagger is right 90% of the time on kaons, one tagged track in three is a pion, because pions are nine times more common. The prior $P(K)$ matters as much as the likelihood $P(T \mid K)$; Note 6 builds classifiers on exactly this computation.

1.2 Expectation, variance and covariance

Expectations are integrals against the density, $\mathbb{E}[g(\mathbf{X})] = \int g(\mathbf{x})\, p(\mathbf{x})\, d\mathbf{x}$ (a sum for discrete variables). The mean is $\boldsymbol{\mu} = \mathbb{E}[\mathbf{X}]$, and the covariance matrix is

$$\Sigma = \mathrm{Cov}(\mathbf{X}) = \mathbb{E}\big[(\mathbf{X}-\boldsymbol{\mu})(\mathbf{X}-\boldsymbol{\mu})^{\mathsf{T}}\big], \qquad \Sigma_{jk} = \mathbb{E}[(X_j-\mu_j)(X_k-\mu_k)],$$

with the variances $\mathrm{Var}(X_j) = \Sigma_{jj}$ on its diagonal. The *correlation coefficient* $\rho_{jk} = \Sigma_{jk}/\sqrt{\Sigma_{jj}\Sigma_{kk}}$ lies in $[-1, 1]$ and measures linear dependence only. Three rules are used throughout the course. The first is linearity, $\mathbb{E}[A\mathbf{X} + \mathbf{b}] = A\,\mathbb{E}[\mathbf{X}] + \mathbf{b}$ for a fixed matrix A and vector $\mathbf{b}$, because an integral is linear. Expanding the product in the definition and using linearity gives the useful shortcuts $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$ and $\mathrm{Cov}(X, Z) = \mathbb{E}[XZ] - \mathbb{E}[X]\,\mathbb{E}[Z]$. The second is how a covariance changes under a linear map. Let $\mathbf{Y} = A\mathbf{X} + \mathbf{b}$. Then, step by step,

$$\begin{aligned}\mathbf{Y} - \mathbb{E}[\mathbf{Y}] &= A\mathbf{X} + \mathbf{b} - (A\boldsymbol{\mu} + \mathbf{b}) = A(\mathbf{X} - \boldsymbol{\mu}),\\ \mathrm{Cov}(\mathbf{Y}) &= \mathbb{E}\big[A(\mathbf{X}-\boldsymbol{\mu})\,(A(\mathbf{X}-\boldsymbol{\mu}))^{\mathsf{T}}\big] = \mathbb{E}\big[A(\mathbf{X}-\boldsymbol{\mu})(\mathbf{X}-\boldsymbol{\mu})^{\mathsf{T}}A^{\mathsf{T}}\big] = A\,\mathbb{E}\big[(\mathbf{X}-\boldsymbol{\mu})(\mathbf{X}-\boldsymbol{\mu})^{\mathsf{T}}\big]\,A^{\mathsf{T}},\end{aligned}$$

so

$$\mathrm{Cov}(A\mathbf{X} + \mathbf{b}) = A\,\Sigma\,A^{\mathsf{T}}. \tag{1}$$

In one dimension this is the familiar $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$: a shift does not change the spread and a scale multiplies it by a^2. The third rule follows from (1) with $A = \mathbf{v}^{\mathsf{T}}$ a single row: for any $\mathbf{v}$, $\mathrm{Var}(\mathbf{v}^{\mathsf{T}}\mathbf{X}) = \mathbf{v}^{\mathsf{T}}\Sigma\mathbf{v} \geq 0$. A covariance matrix is therefore symmetric and *positive semi-definite*; it is positive definite unless some combination $\mathbf{v}^{\mathsf{T}}\mathbf{X}$ is constant. Finally, independent variables are uncorrelated, but uncorrelated variables need not be independent: if X is uniform on $[-1, 1]$ and $Z = X^2$, then $\mathbb{E}[X] = 0$ and $\mathrm{Cov}(X, Z) = \mathbb{E}[XZ] - \mathbb{E}[X]\,\mathbb{E}[Z] = \mathbb{E}[X^3] = 0$, because x^3 is odd and X is symmetric about 0; yet Z is a function of X.

2 Four things we do with a distribution

Whether the distribution is a probability table, a Gaussian or a neural network with billions of parameters, the same four operations recur.

1. **Sample**: generate new data $\mathbf{x} \sim p$. This is what a generative model is for.
2. **Evaluate**: compute $p(\mathbf{x})$, the density (or probability) the model assigns to an observation, summing or integrating out anything that was not recorded. Viewed as a function of the model parameters for fixed data, the same number is called the *likelihood*. Low likelihood flags unusual data.
3. **Infer**: predict the hidden part of a configuration from the observed part, for example by its single most likely value or by its average $\mathbb{E}[\mathbf{Z} \mid \mathbf{X} = \mathbf{x}]$. Supervised learning is inference of Y from $\mathbf{X}$.
4. **Learn**: choose the parameters θ of a family p_θ from data. The standard principle is *maximum likelihood*,

$$\hat{\theta} = \arg\max_{\theta} \prod_{i=1}^{n} p_\theta(\mathbf{x}_i) = \arg\min_{\theta}\; -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(\mathbf{x}_i), \tag{2}$$

the product being justified by the i.i.d. assumption. Taking the logarithm changes nothing about the maximiser, because log is increasing, but turns a product of n small numbers (which underflows) into a sum (which does not) and makes derivatives easy. The right-hand side is the average negative log-likelihood, the density-estimation loss of Note 1, Table 2.

Worked example — a categorical distribution learned from counts

The number $X \in \{0, 1, 2\}$ of reconstructed jets in a simplified event selection is described by three numbers $\pi_0, \pi_1, \pi_2 \geq 0$ with $\sum_k \pi_k = 1$ — a *categorical* distribution with $m = 3$ categories. In $n = 50$ selected events we count $n_0 = 12$, $n_1 = 30$, $n_2 = 8$.

- *Learn.* Maximum likelihood gives the observed frequencies (derived in Section 2.1): $\hat{\pi}_0 = 12/50 = 0.24$, $\hat{\pi}_1 = 30/50 = 0.60$, $\hat{\pi}_2 = 8/50 = 0.16$ (Figure 1, left).
- *Evaluate.* The log-likelihood of the data is $\sum_k n_k \log \hat{\pi}_k = 12 \log 0.24 + 30 \log 0.60 + 8 \log 0.16 \approx -47.11$, i.e. an average negative log-likelihood of 0.942 nats per event (nats are natural-logarithm units; dividing by $\ln 2$ gives bits). The uniform guess $\pi_k = 1/3$ gives $50 \log(1/3) \approx -54.93$: the fitted model explains the data better, as it must.
- *Sample.* The c.d.f. is $F(0) = 0.24$, $F(1) = 0.84$, $F(2) = 1$. Draw u uniform on $[0, 1)$ and return the first k with $F(k) > u$: $u = 0.10$ gives $k = 0$, $u = 0.55$ gives $k = 1$ and $u = 0.90$ gives $k = 2$ (Figure 1, right). Each k is returned with probability equal to the length of its step, $\hat{\pi}_k$.
- *Infer.* The most probable multiplicity is 1, and the mean is $0{\times}0.24 + 1{\times}0.60 + 2{\times}0.16 = 0.92$ jets.

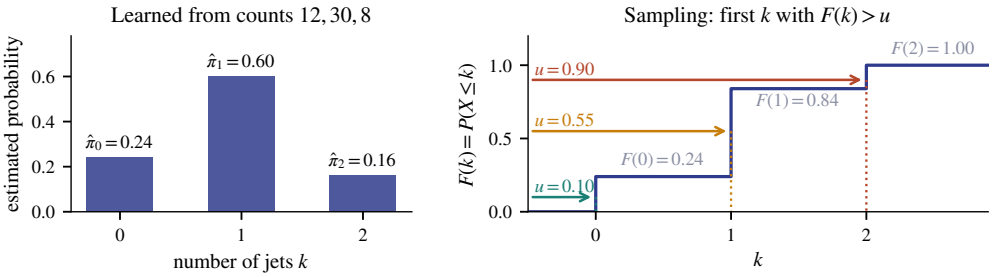


Figure 1. The categorical example. Left: maximum-likelihood probabilities from the counts 12, 30, 8. Right: inverse-c.d.f. sampling. A uniform u is sent horizontally until it meets the staircase F; the step it hits is the sample.

2.1 Maximum likelihood for a categorical distribution

With m categories and counts $n_1, \dots, n_m$ ($\sum_k n_k = n$), the probability of the data is $\prod_i \pi_{x_i} = \prod_k \pi_k^{n_k}$, so the log-likelihood is

$$\ell(\pi) = \sum_{k=1}^{m} n_k \log \pi_k, \qquad \text{to be maximised subject to } \sum_{k=1}^{m} \pi_k = 1.$$

The constraint matters: without it, ℓ grows without bound as every π_k grows. A constrained maximum is found with a *Lagrange multiplier* λ. Form

$$\mathscr{L}(\pi, \lambda) = \sum_k n_k \log \pi_k + \lambda\Big(1 - \sum_k \pi_k\Big)$$

and set its derivatives to zero:

$$\frac{\partial \mathscr{L}}{\partial \pi_k} = \frac{n_k}{\pi_k} - \lambda = 0 \quad \Longrightarrow \quad \pi_k = \frac{n_k}{\lambda} \quad (k = 1, \dots, m),$$

$$\frac{\partial \mathscr{L}}{\partial \lambda} = 1 - \sum_k \pi_k = 0 \quad \Longrightarrow \quad 1 = \sum_k \frac{n_k}{\lambda} = \frac{n}{\lambda} \quad \Longrightarrow \quad \lambda = n.$$

Hence

$$\hat{\pi}_k = \frac{n_k}{n}. \tag{3}$$

This stationary point is the maximum: each $\log \pi_k$ is concave, so ℓ is concave, and a concave function on the convex set of probability vectors has no other local maxima. In the language of losses, dividing ℓ by n gives $\sum_k \hat{p}_k \log \pi_k$ with $\hat{p}_k = n_k/n$, so maximum likelihood minimises the *cross-entropy* $-\sum_k \hat{p}_k \log \pi_k$ between the observed frequencies and the model (Note 4, Section 2), and the minimum is reached when the model reproduces the frequencies.

Zero counts. If a category never occurs, $n_k = 0$ and $\hat{\pi}_k = 0$: the model declares the category impossible. A single later occurrence then has log-likelihood $\log 0 = -\infty$. The standard remedy adds a *pseudo-count* $\alpha > 0$ to every category,

$$\hat{\pi}_k^{(\alpha)} = \frac{n_k + \alpha}{n + \alpha m}, \tag{4}$$

which with $\alpha = 1$ (Laplace smoothing) turns the jet counts into the probabilities $13/53$, $31/53$ and $9/53$, that is 0.245, 0.585 and 0.170. The correction matters for small n_k and fades as n grows; Note 6, Section 4, uses it in naive Bayes.

2.2 Sampling from a categorical distribution

The sampling rule of the worked example is the *inverse-c.d.f. method*. Number the categories $1, \dots, m$ (in the jet example, category k is $k-1$ jets) and set $F(0) = 0$. If U is uniform on $[0, 1)$, the returned value is k exactly when $F(k-1) \le U < F(k)$, which happens with probability $F(k) - F(k-1) = \pi_k$. For unordered categories the numbering is arbitrary, and any numbering works, because each step of the staircase still has length π_k. The same idea samples a continuous X with an invertible c.d.f.: the variable $F^{-1}(U)$ satisfies $P(F^{-1}(U) \le x) = P(U \le F(x)) = F(x)$, so a draw is $x = F^{-1}(u)$. For example, the exponential decay-time density $p(x) = e^{-x/\tau}/\tau$ has $F(x) = 1 - e^{-x/\tau}$, so $x = -\tau \log(1-u)$. Scanning the categories one by one costs up to m steps per draw. When many samples are needed from the same distribution, the cumulative sums can be computed once and the first k with $F(k) > u$ found by *binary search*, because F is sorted.

Worked example — binary search on a staircase

Take $m = 8$ categories with cumulative sums $F_{1:8} = (0.05, 0.15, 0.30, 0.40, 0.55, 0.75, 0.90, 1)$ and $u = 0.70$. SampleBinary of Algorithm 1 starts with $lo = 1$, $hi = 8$.

Step	lo	hi	mid	F_{mid}	action
1	1	8	4	0.40	$0.40 \le u$: $lo \leftarrow 5$
2	5	8	6	0.75	$0.75 > u$: $hi \leftarrow 6$
3	5	6	5	0.55	$0.55 \le u$: $lo \leftarrow 6$

Now $lo = hi = 6$, and indeed $F_5 = 0.55 \le 0.70 < 0.75 = F_6$. Three comparisons, $\log_2 8$, where the linear scan would have needed six.

Algorithm 1 The categorical distribution: fitting by counting; sampling by inverse c.d.f.

Require: labels $x_1, \dots, x_n \in \{1, \dots, m\}$; pseudo-count $\alpha \geq 0$ ($\alpha = 0$ is maximum likelihood)

function FitCategorical($x_{1:n}, m, \alpha$)
  $c_k \leftarrow 0$ for $k = 1, \dots, m$
  for $i = 1, \dots, n$ **do** $c_{x_i} \leftarrow c_{x_i} + 1$ ▷ one pass over the data
  end for
  return $\hat{\pi}_k = (c_k + \alpha)/(n + \alpha m)$ for $k = 1, \dots, m$ ▷ equation (4)
end function

Sampling from probabilities $\pi_1, \dots, \pi_m$ (non-negative, sum one):

function SampleLinear(π)
  draw $u \sim \text{Uniform}[0, 1)$; $F \leftarrow 0$ ▷ running c.d.f.
  for $k = 1, \dots, m$ **do**
    $F \leftarrow F + \pi_k$
    if $F > u$ **then return** k
    end if
  end for
  return m ▷ guards against round-off in $\sum_k \pi_k$
end function
function SampleBinary($F_{1:m}$) ▷ $F_k = \pi_1 + \cdots + \pi_k$, computed once in $O(m)$
  draw $u \sim \text{Uniform}[0, 1)$; $lo \leftarrow 1$; $hi \leftarrow m$
  while $lo < hi$ **do** ▷ invariant: the answer lies in $\{lo, \dots, hi\}$
    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$
    if $F_{mid} > u$ **then** $hi \leftarrow mid$
    else $lo \leftarrow mid + 1$
    end if
  end while
  return lo
end function

Running time — fitting and sampling a categorical distribution

Fitting: $O(n+m)$ time — one increment per example plus one division per category — and $O(m)$ memory for the counts; the data can be streamed and need not be stored. *Sampling, linear scan*: $O(m)$ per draw in the worst case, $O(1)$ extra memory. *Sampling, binary search*: $O(m)$ once to build F, then $O(\log m)$ per draw, because each step halves the interval $\{lo, \dots, hi\}$; memory $O(m)$. For $m = 10^5$ (a language-model vocabulary) a draw takes about 17 comparisons instead of up to 10^5. The alias method even reaches $O(1)$ per draw after $O(m)$ preparation. In a language model π changes at every token, so producing it (the network, then an $O(m)$ softmax) dominates, and the $O(m)$ cumulative sum makes binary search pointless for a single draw. (The softmax normalisation is defined in the box below.)

Modern connection — a language model is a categorical distribution

A large language model defines, for every prefix of text, a categorical distribution over a vocabulary of $V \approx 10^5$ tokens (words or pieces of words). The network outputs one real score per token, the "logits" $z_1, \dots, z_V$, and the probabilities are given by the *softmax* $\pi_k = e^{z_k/T} / \sum_j e^{z_j/T}$, where the *temperature* $T = 1$ during training. All four operations appear: training is maximum likelihood (2) on the next token, i.e. minimising cross-entropy; generating text is repeated sampling (Algorithm 1), with $T < 1$ concentrating mass on likely tokens, $T \to 0$ giving the single most probable token and $T \to \infty$ approaching the uniform distribution; evaluating the likelihood of a text gives its *perplexity*, exp of the average negative log-likelihood per token; and answering a question is inference, conditioning on the prompt. The worked example above — learned frequencies from counts, with perplexity $e^{0.942} \approx 2.57$ — is the simplest version of

this. Counting next-token frequencies separately for every possible prefix would repeat that fit once per prefix, but almost every long prefix occurs zero times in the training text, so counting fails. A neural network instead uses the same weights for all prefixes, so what it learns from one prefix also informs similar ones.

3 The univariate Gaussian

The Gaussian (normal) density with mean μ and variance σ^2 is

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{5}$$

It is a symmetric bump centred at μ with width set by the standard deviation σ (Figure 2). About 68.3%, 95.4% and 99.7% of the probability lies within one, two and three standard deviations of the mean; physicists' "5σ" discovery threshold corresponds to a one-sided tail probability of 2.9×10^{-7}. Any Gaussian is a shifted and scaled standard one: $X = \mu + \sigma Z$ with $Z \sim \mathcal{N}(0,1)$.

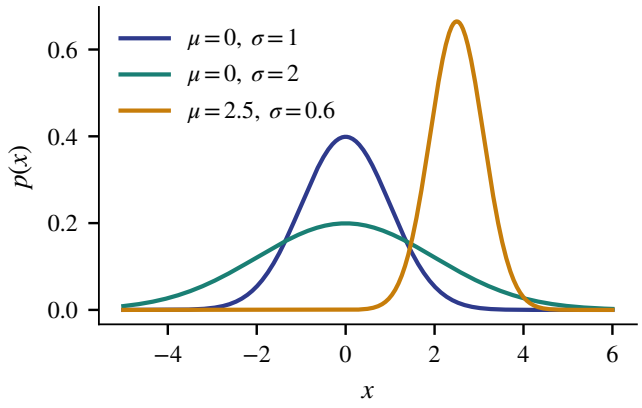


Figure 2. Gaussian densities. Doubling σ halves the peak height, since the area stays one.

Why this particular shape. The central limit theorem says that a sum of many independent contributions with finite variance is approximately Gaussian, whatever the individual distributions, provided no single contribution dominates the sum. Measurement noise built from many small effects is therefore often close to Gaussian. A second reason is that, among all densities with a given mean and variance, the Gaussian has maximal entropy $H = -\int p \log p \, dx$ (the information-theoretic analogue of the Gibbs entropy): it assumes nothing beyond those two numbers.

3.1 Standardising and computing probabilities

The Gaussian c.d.f. cannot be written with elementary functions. Every Gaussian probability is instead reduced to the standard c.d.f. $\Phi(z) = P(Z \le z)$ by *standardising*: if $X = \mu + \sigma Z$, then $X \le x$ exactly when $Z \le (x-\mu)/\sigma$, so

$$P(X \le x) = \Phi\!\left(\frac{x-\mu}{\sigma}\right), \qquad z = \frac{x-\mu}{\sigma} \text{ is the } z\text{-score of } x.$$

The symmetry of the bump gives $\Phi(-z) = 1-\Phi(z)$, and a two-sided interval has $P(|Z| \le z) = 2\Phi(z)-1$. A handful of values suffices for most purposes:

z	0	0.5	1	1.5	1.645	1.96	2	3
$\Phi(z)$	0.5000	0.6915	0.8413	0.9332	0.9500	0.9750	0.9772	0.9987
$2\Phi(z)-1$	0	0.383	0.683	0.866	0.900	0.950	0.954	0.997

The last row is the "68–95–99.7" rule, and the columns 1.645 and 1.96 give the familiar 90% and 95% intervals.

Worked example — probabilities for a calibrated energy measurement

Suppose a simple calorimeter measures the energy of 50 GeV electrons with Gaussian resolution, $X \sim \mathcal{N}(50, 2^2)$ in GeV (a deliberately coarse, hypothetical resolution).

- $P(X > 54)$: the z-score is $(54-50)/2 = 2$, so $P = 1 - \Phi(2) = 1 - 0.9772 = 0.0228$. About one electron in 44 is measured above 54 GeV.
- $P(47 < X < 53)$: $z = \pm 1.5$, so $P = 2\Phi(1.5) - 1 = 0.866$.
- $P(X < 49)$: $z = -0.5$, so $P = \Phi(-0.5) = 1 - 0.6915 = 0.3085$.
- The interval containing the central 90% is $50 \pm 1.645 \times 2 = [46.71, 53.29]$ GeV.

The peak density is $1/\sqrt{2\pi \cdot 4} \approx 0.199$ GeV^{-1}; multiplying by a bin width of 0.1 GeV gives the approximate probability 0.020 of a measurement within ± 0.05 GeV of 50.

3.2 Gaussian noise and squared error

Suppose targets are generated as $y = f(x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, independently for each example. Then $y \mid x \sim \mathcal{N}(f(x), \sigma^2)$ and, taking $-\log$ of (5),

$$-\log p(y \mid x) = \frac{(y - f(x))^2}{2\sigma^2} + \frac{1}{2}\log(2\pi\sigma^2).$$

Summed over a dataset, this is $\frac{1}{2\sigma^2}\sum_i (y_i - f(x_i))^2 + \frac{n}{2}\log(2\pi\sigma^2)$. The second term does not depend on f, and the factor $1/(2\sigma^2)$ is a positive constant, so the f that maximises the likelihood is the f that minimises the sum of squared errors, whatever the value of σ. The squared loss of Note 1, Table 2, is maximum likelihood under Gaussian noise; Note 8, Section 2, builds least squares on this fact, and heavier-tailed noise calls for other losses (Question C3).

In physics — χ^2 fits are Gaussian likelihoods

Fitting a model f_θ to measurements $y_i \pm \sigma_i$ by minimising $\chi^2 = \sum_i (y_i - f_\theta(x_i))^2/\sigma_i^2$ is maximum likelihood with independent Gaussian errors of known widths: the derivation above with a different σ_i for each point gives $-2\log$ likelihood $= \chi^2 +$ const. The familiar rule "$\Delta\chi^2 = 1$ gives the 1σ interval" of a single parameter is the statement that $-2\log$ likelihood is quadratic near its minimum [3]. When errors are correlated, the sum becomes the quadratic form of the next section, with the covariance matrix of the measurements.

4 The multivariate Gaussian

For $\mathbf{x} \in \mathbb{R}^d$, the Gaussian with mean $\boldsymbol{\mu} \in \mathbb{R}^d$ and symmetric positive-definite covariance matrix $\Sigma \in \mathbb{R}^{d\times d}$ is

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\!\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big), \tag{6}$$

where $|\Sigma|$ is the determinant. For $d = 1$ it reduces to (5). The prefactor only normalises the density; all the shape is in the exponent. The quantity

$$\Delta^2(\mathbf{x}) = (\mathbf{x}-\boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})$$

is the squared *Mahalanobis distance* of $\mathbf{x}$ from the mean: Euclidean distance after the data have been rotated to the principal axes and rescaled so that every direction has unit variance ("whitened"; see the eigendecomposition below). Surfaces of constant density are the ellipsoids $\Delta^2 =$ const. If the

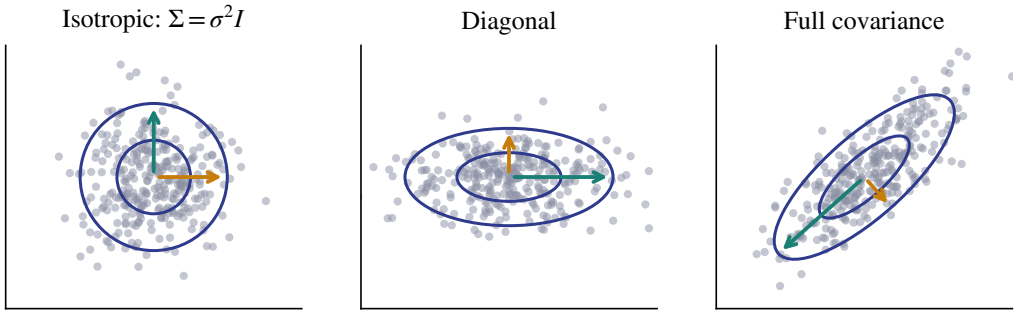


Figure 3. Samples (grey) and the one- and two-σ contour ellipses of three Gaussians in $d = 2$. Arrows are the eigenvectors scaled by $2\sqrt{\lambda_i}$. The isotropic case has no preferred direction; the full covariance captures a correlation between the two coordinates.

components are independent, $\Sigma = \mathrm{diag}(\sigma_1^2, \dots, \sigma_d^2)$, then $\Delta^2 = \sum_j (x_j - \mu_j)^2/\sigma_j^2$ is a sum of squared z-scores and the density is the product of d one-dimensional Gaussians, $\prod_j \mathcal{N}(x_j; \mu_j, \sigma_j^2)$. In the *isotropic* case $\Sigma = \sigma^2 I$ we have $|\Sigma| = \sigma^{2d}$ and $\Sigma^{-1} = I/\sigma^2$, so

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \sigma^2 I) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\!\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}\|^2}{2\sigma^2}\right) : \tag{7}$$

the Mahalanobis distance is the Euclidean distance divided by σ, and the contours are spheres. For $d = 2$ and $\sigma = 1$ the peak density is $1/(2\pi) \approx 0.159$.

Linear maps keep a Gaussian Gaussian. A property used repeatedly below: if $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and A is a fixed $k \times d$ matrix of rank k, then

$$A\mathbf{X} + \mathbf{b} \sim \mathcal{N}\big(A\boldsymbol{\mu} + \mathbf{b},\ A\Sigma A^{\mathsf{T}}\big). \tag{8}$$

The mean and covariance are those of (1); that the result is again Gaussian follows, for invertible A, from a change of variables in (6), and in general is a standard result [1]. Keeping some coordinates of $\mathbf{X}$ is such a map, which is why marginals of a Gaussian are Gaussian (Section 4.3).

Geometry through the eigendecomposition. Write $\Sigma = \sum_{i=1}^{d} \lambda_i \mathbf{v}_i \mathbf{v}_i^{\mathsf{T}}$ with orthonormal eigenvectors $\mathbf{v}_i$ and eigenvalues $\lambda_i > 0$. The inverse has the same eigenvectors and inverted eigenvalues, $\Sigma^{-1} = \sum_i \lambda_i^{-1} \mathbf{v}_i \mathbf{v}_i^{\mathsf{T}}$ (multiply the two sums and use $\mathbf{v}_i^{\mathsf{T}} \mathbf{v}_j = 0$ for $i \neq j$, $= 1$ for $i = j$). In matrix form $\Sigma = V \Lambda V^{\mathsf{T}}$, where V has the $\mathbf{v}_i$ as columns and $\Lambda = \mathrm{diag}(\lambda_1, \dots, \lambda_d)$; we also write $\Lambda^{1/2} = \mathrm{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_d})$. In the rotated coordinates $u_i = \mathbf{v}_i^{\mathsf{T}}(\mathbf{x} - \boldsymbol{\mu})$ the exponent therefore becomes

$$(\mathbf{x} - \boldsymbol{\mu})^{\mathsf{T}} \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) = \sum_{i=1}^{d} \frac{\big(\mathbf{v}_i^{\mathsf{T}}(\mathbf{x} - \boldsymbol{\mu})\big)^2}{\lambda_i} = \sum_{i=1}^{d} \frac{u_i^2}{\lambda_i},$$

so the density factorises into d independent one-dimensional Gaussians with variances λ_i. This also explains the prefactor of (6): the rotation $\mathbf{u} = V^{\mathsf{T}}(\mathbf{x} - \boldsymbol{\mu})$ preserves volumes (its Jacobian is 1), so

$$\int_{\mathbb{R}^d} e^{-\Delta^2(\mathbf{x})/2}\, d\mathbf{x} = \prod_{i=1}^{d} \int_{-\infty}^{\infty} e^{-u_i^2/(2\lambda_i)}\, du_i = \prod_{i=1}^{d} \sqrt{2\pi\lambda_i} = (2\pi)^{d/2} |\Sigma|^{1/2}.$$

The ellipsoid $\Delta = 1$ has its axes along the eigenvectors with half-lengths $\sqrt{\lambda_i}$, and $|\Sigma| = \prod_i \lambda_i$ is proportional to its squared volume (the volume is $V_d \sqrt{|\Sigma|}$, with V_d the volume of the unit ball; $\pi\sqrt{|\Sigma|}$ in $d = 2$). Three families are common (Figure 3):

Family	Covariance	Free parameters in Σ	Shape
Isotropic	$\sigma^2 I$	1	spheres
Diagonal	$\mathrm{diag}(\sigma_1^2, \dots, \sigma_d^2)$	d	axis-aligned ellipsoids
Full	any positive-definite Σ	$d(d+1)/2$	arbitrary ellipsoids

The choice is a choice of model size. For $d = 1000$ a full covariance has about half a million parameters, which $n = 500$ examples cannot determine; Note 4, Section 5, calls this a question of *capacity*.

4.1 A two-dimensional example with numbers

One covariance matrix is worked through in full here; the same numbers return for conditioning (Section 4.3), sampling (Section 5) and log-densities (Section 6.5), so that every formula can be checked by hand.

Worked example — eigenvalues, ellipse axes and Mahalanobis distance

Take

$$\boldsymbol{\mu} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}.$$

The variances are 5 and 2 and the correlation is $\rho = 2/\sqrt{5 \cdot 2} \approx 0.63$.
Eigenvalues. They solve $\det(\Sigma - \lambda I) = (5-\lambda)(2-\lambda) - 4 = \lambda^2 - 7\lambda + 6 = 0$, so $\lambda_1 = 6$ and $\lambda_2 = 1$. Check: $\lambda_1 + \lambda_2 = 7 = \mathrm{tr}\,\Sigma$ and $\lambda_1 \lambda_2 = 6 = |\Sigma|$.
Eigenvectors. For $\lambda_1 = 6$, the first row of $(\Sigma - 6I)\mathbf{v} = 0$ reads $-v_1 + 2v_2 = 0$, so $\mathbf{v}_1 = (2,1)/\sqrt{5}$. For $\lambda_2 = 1$, $4v_1 + 2v_2 = 0$ gives $\mathbf{v}_2 = (1,-2)/\sqrt{5}$. They are orthogonal, as they must be for a symmetric matrix.
Ellipse. The contour $\Delta = 1$ is an ellipse centred at $(1,2)$ with long half-axis $\sqrt{6} \approx 2.45$ along $(2,1)$ — tilted by $\arctan(1/2) \approx 26.6^\circ$ — and short half-axis $\sqrt{1} = 1$ along $(1,-2)$. The contour $\Delta = 2$ is the same ellipse scaled by two (Figure 4, left). The area inside $\Delta = 1$ is $\pi\sqrt{\lambda_1 \lambda_2} = \pi\sqrt{|\Sigma|} = \pi\sqrt{6} \approx 7.70$.
Mahalanobis distance. The inverse is $\Sigma^{-1} = \frac{1}{6}\begin{pmatrix} 2 & -2 \\ -2 & 5 \end{pmatrix}$, so for a displacement $\mathbf{r} = \mathbf{x} - \boldsymbol{\mu}$ the quadratic form is $\Delta^2 = \mathbf{r}^{\mathsf{T}}\Sigma^{-1}\mathbf{r} = (2r_1^2 - 4r_1 r_2 + 5r_2^2)/6$ (the cross term appears twice). Compare two points at the *same* Euclidean distance $\sqrt{5} \approx 2.24$ from the mean:

$$A = (3,3): \quad \mathbf{r} = (2,1), \qquad \Delta^2 = \tfrac{1}{6}(2 \cdot 4 - 4 \cdot 2 \cdot 1 + 5 \cdot 1) = \tfrac{5}{6}, \qquad \Delta_A \approx 0.91,$$
$$B = (2,0): \quad \mathbf{r} = (1,-2), \qquad \Delta^2 = \tfrac{1}{6}\big(2 \cdot 1 - 4 \cdot 1 \cdot (-2) + 5 \cdot 4\big) = 5, \qquad \Delta_B \approx 2.24.$$

The eigenvector form gives the same numbers: $\mathbf{r}_A = (2,1) = \sqrt{5}\,\mathbf{v}_1$, so $\Delta^2 = 5/\lambda_1 = 5/6$, while $\mathbf{r}_B = (1,-2) = \sqrt{5}\,\mathbf{v}_2$, so $\Delta^2 = 5/\lambda_2 = 5$. Point A lies along the long axis, where the distribution is wide, and is unremarkable; B lies along the short axis and is more than two "standard deviations" out. The densities differ by a factor $e^{-(5-5/6)/2} \approx 0.12$: $\mathcal{N}(A) = e^{-5/12}/(2\pi\sqrt{6}) \approx 0.0428$ and $\mathcal{N}(B) = e^{-5/2}/(2\pi\sqrt{6}) \approx 0.0053$.

4.2 How much probability lies inside an ellipse

In one dimension, 68% of the probability lies within $\Delta = |x - \mu|/\sigma \le 1$. This rule does not carry over to higher dimension. In the rotated and rescaled coordinates $w_i = u_i/\sqrt{\lambda_i}$ the w_i are independent standard normals and $\Delta^2 = \sum_i w_i^2$, a sum of d squared standard normals: a χ^2 variable with d degrees of freedom. In $d = 2$ the probability can be computed in polar coordinates (r, φ) for (w_1, w_2). The joint

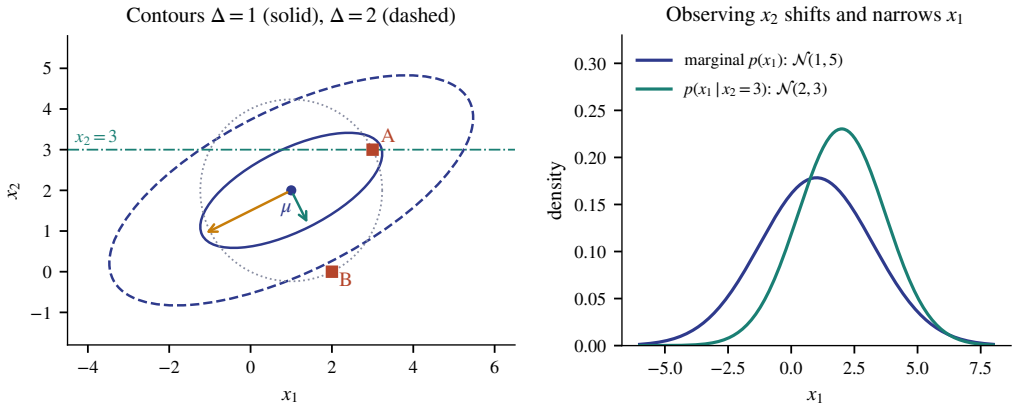


Figure 4. The worked examples of Sections 4.1–4.3. Left: contours of $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ with $\boldsymbol{\mu} = (1, 2)$, $\Sigma = \left(\begin{smallmatrix} 5 & 2 \\ 2 & 2 \end{smallmatrix}\right)$; arrows are the eigenvectors scaled by $\sqrt{\lambda_i}$; the dotted circle has Euclidean radius $\sqrt{5}$ and passes through both A and B. Right: the marginal of x_1 and its conditional on the line $x_2 = 3$.

density of (w_1, w_2) is $\frac{1}{2\pi} e^{-(w_1^2 + w_2^2)/2}$, the region $\Delta \le k$ is the disc $w_1^2 + w_2^2 \le k^2$, and $dw_1\, dw_2 = r\, dr\, d\varphi$, so

$$P(\Delta \le k) = \int_0^{2\pi} \int_0^k \frac{1}{2\pi} e^{-r^2/2}\, r\, dr\, d\varphi = \frac{2\pi}{2\pi} \int_0^k r\, e^{-r^2/2}\, dr = \Big[-e^{-r^2/2} \Big]_0^k = 1 - e^{-k^2/2}.$$

The ellipses $\Delta = 1, 2, 3$ therefore contain 39.3%, 86.5% and 98.9% of the probability — less than in one dimension — and the 95% ellipse is $\Delta = \sqrt{-2 \log 0.05} \approx 2.45$. As d grows, the mass moves further out: $\mathbb{E}[\Delta^2] = d$, and the typical sample lies at $\Delta \approx \sqrt{d}$ (Question C4; Note 3, Section 3).

4.3 Marginals and conditionals

Closed under marginalising and conditioning. Split $\mathbf{x} = (\mathbf{x}_a, \mathbf{x}_b)$ into d_a coordinates to be predicted and d_b observed ones, and partition the mean and covariance in the same way,

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}, \qquad \Sigma_{ba} = \Sigma_{ab}^{\mathsf{T}},$$

where $\Sigma_{ab} = \mathrm{Cov}(\mathbf{x}_a, \mathbf{x}_b)$ is $d_a \times d_b$. In the example of Section 4.1 with $a = \{1\}$ and $b = \{2\}$, $\Sigma_{aa} = 5$, $\Sigma_{ab} = \Sigma_{ba} = 2$ and $\Sigma_{bb} = 2$. Every marginal of a Gaussian is Gaussian — just keep the corresponding entries, $\mathbf{x}_a \sim \mathcal{N}(\boldsymbol{\mu}_a, \Sigma_{aa})$ — and every conditional is Gaussian,

$$\mathbf{x}_a \mid \mathbf{x}_b \sim \mathcal{N}\big(\boldsymbol{\mu}_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\mathbf{x}_b - \boldsymbol{\mu}_b),\ \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\big). \tag{9}$$

The conditional mean is linear in the observed part and the conditional covariance does not depend on it. Since a Gaussian density peaks at its mean, the conditional mean is also the single most probable value of $\mathbf{x}_a$, so both readings of "infer" in Section 2 give the same answer. This is inference in closed form: if two detector channels are correlated, reading one sharpens the prediction for the other by an amount set by the correlation. Formula (9) underlies the Kalman filter used for particle-track reconstruction and spacecraft navigation, and Gaussian-process regression.

Where the formula comes from (two dimensions). Let (X_1, X_2) be jointly Gaussian with variances Σ_{11}, Σ_{22} and covariance Σ_{12}. Split X_1 into a part explained by X_2 and a remainder,

$$X_1 = \mu_1 + \beta\,(X_2 - \mu_2) + E, \qquad \beta = \frac{\Sigma_{12}}{\Sigma_{22}}.$$

This only *defines* $E = X_1 - \beta X_2 + \text{const}$. Its mean is zero, and because covariance is linear in each argument (and ignores constants), its covariance with X_2 is

$$\operatorname{Cov}(E, X_2) = \operatorname{Cov}(X_1, X_2) - \beta \operatorname{Var}(X_2) = \Sigma_{12} - \frac{\Sigma_{12}}{\Sigma_{22}}\Sigma_{22} = 0.$$

E is a linear function of the Gaussian vector (X_1, X_2), so (E, X_2) is jointly Gaussian by (8), and for jointly Gaussian variables zero covariance implies independence (the density factorises because Σ is then diagonal). Knowing $X_2 = x_2$ thus leaves the distribution of E unchanged, and

$$\mathbb{E}[X_1 \mid X_2 = x_2] = \mu_1 + \frac{\Sigma_{12}}{\Sigma_{22}}(x_2 - \mu_2),$$

$$\operatorname{Var}(X_1 \mid X_2 = x_2) = \operatorname{Var}(E) = \operatorname{Var}(X_1) - 2\beta \operatorname{Cov}(X_1, X_2) + \beta^2 \operatorname{Var}(X_2) = \Sigma_{11} - \frac{\Sigma_{12}^2}{\Sigma_{22}},$$

which is (9) for $d = 2$. The variance of $E = X_1 - \beta X_2 + \text{const}$ is (1) with the row $A = (1, -\beta)$: $(1, -\beta)\,\Sigma\,(1, -\beta)^{\mathsf{T}} = \Sigma_{11} - 2\beta\Sigma_{12} + \beta^2\Sigma_{22}$, and inserting $\beta = \Sigma_{12}/\Sigma_{22}$ gives the last expression. In terms of the correlation, the conditional variance is $\Sigma_{11}(1 - \rho^2)$: observing X_2 removes the fraction ρ^2 of the variance of X_1. The same argument with matrices, $B = \Sigma_{ab}\Sigma_{bb}^{-1}$ in place of β, gives the general case (Question C5).

Worked example — conditioning the two-dimensional example

For $\boldsymbol{\mu} = (1, 2)$ and $\Sigma = \left(\begin{smallmatrix} 5 & 2 \\ 2 & 2 \end{smallmatrix}\right)$, suppose we observe $X_2 = 3$. Then

$$\mathbb{E}[X_1 \mid X_2 = 3] = 1 + \frac{2}{2}(3 - 2) = 2, \qquad \operatorname{Var}(X_1 \mid X_2 = 3) = 5 - \frac{2^2}{2} = 3.$$

Before the observation $X_1 \sim \mathcal{N}(1, 5)$; after it $X_1 \sim \mathcal{N}(2, 3)$ (Figure 4, right). The observed x_2 lies one unit above its mean; with the slope $\beta = \Sigma_{12}/\Sigma_{22} = 1$ this pulls the prediction of x_1 up by one unit, and the variance shrinks by the fraction $\rho^2 = 4/10 = 0.4$. The roles can be reversed: given $X_1 = 3$, $X_2 \sim \mathcal{N}(2 + \frac{2}{5} \cdot 2,\ 2 - \frac{4}{5}) = \mathcal{N}(2.8, 1.2)$.

In computation, Σ_{bb}^{-1} is never formed: with the Cholesky factor of Section 5, $\Sigma_{bb} = L_b L_b^{\mathsf{T}}$, both products in (9) reduce to triangular solves, since $\Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba} = W^{\mathsf{T}}W$ with $W = L_b^{-1}\Sigma_{ba}$ (Algorithm 2). For the example above, $L_b = \sqrt{2}$, $W = 2/\sqrt{2} = \sqrt{2}$ and $\mathbf{u} = (3 - 2)/\sqrt{2}$, so the mean is $1 + Wu = 2$ and the variance $5 - W^2 = 3$, as before.

Algorithm 2 Conditioning a Gaussian on observed coordinates $\mathbf{x}_b$

Require: blocks $\boldsymbol{\mu}_a, \boldsymbol{\mu}_b, \Sigma_{aa}, \Sigma_{ab}, \Sigma_{bb}$ (d_a predicted and d_b observed coordinates); observed $\mathbf{x}_b$
Ensure: mean and covariance of $\mathbf{x}_a \mid \mathbf{x}_b$, equation (9)

```
function CONDITION(μ, Σ, x_b)
    L_b ← CHOLESKY(Σ_bb)                          ▷ Algorithm 3
    solve L_b W = Σ_ba for W ∈ R^{d_b × d_a}      ▷ forward substitution, one column at a time
    solve L_b u = x_b − μ_b for u ∈ R^{d_b}
    return μ_a + W^T u,  Σ_aa − W^T W
end function
```

Running time — conditioning a Gaussian

Set-up: $O(d_b^3)$ for the Cholesky factor of Σ_{bb}, $O(d_a d_b^2)$ for the d_a triangular solves that give W, and $O(d_a^2 d_b)$ for the product $W^{\mathsf{T}}W$; memory $O(d_b^2 + d_a d_b + d_a^2)$. *Per new observation* $\mathbf{x}_b$: $O(d_b^2)$ for one solve plus $O(d_a d_b)$ for $W^{\mathsf{T}}\mathbf{u}$, or only $O(d_a d_b)$ if the matrix $B = \Sigma_{ab}\Sigma_{bb}^{-1}$ is stored. The $O(d_b^3)$ term is why exact Gaussian-process regression, which conditions on n observed training targets, costs $O(n^3)$.

5 Sampling: from white noise to anything Gaussian

To sample $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$, factor $\Sigma = LL^{\mathsf{T}}$ (Cholesky decomposition, L lower triangular with positive diagonal), draw $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I)$ — d independent standard normals — and return

$$\mathbf{x} = \boldsymbol{\mu} + L\mathbf{z}. \tag{10}$$

Why this has covariance Σ. The vector $\mathbf{z}$ has mean $\mathbf{0}$ and covariance I, because its components are independent with unit variance. Apply the rules of Section 1.2 with $A = L$ and $\mathbf{b} = \boldsymbol{\mu}$:

$$\mathbb{E}[\mathbf{x}] = L\,\mathbb{E}[\mathbf{z}] + \boldsymbol{\mu} = \boldsymbol{\mu}, \qquad \mathrm{Cov}(\mathbf{x}) = L\ \mathrm{Cov}(\mathbf{z})\, L^{\mathsf{T}} = L\, I\, L^{\mathsf{T}} = LL^{\mathsf{T}} = \Sigma.$$

A linear map of a Gaussian vector is again Gaussian (8), and a Gaussian is fixed by its mean and covariance, so $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$. Any L with $LL^{\mathsf{T}} = \Sigma$ would do — for instance $L = V\Lambda^{1/2}$ from the eigendecomposition $\Sigma = V\Lambda V^{\mathsf{T}}$ of Section 4, which samples independently along each ellipse axis — but $\mathbf{x} = \boldsymbol{\mu} + \Sigma\mathbf{z}$ would not: its covariance is $\Sigma\Sigma^{\mathsf{T}} = \Sigma^2$.

The Cholesky factor by hand. For a 2×2 matrix, multiply out LL^{T} and match entries:

$$\begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix}\begin{pmatrix} L_{11} & L_{21} \\ 0 & L_{22} \end{pmatrix} = \begin{pmatrix} L_{11}^2 & L_{11}L_{21} \\ L_{11}L_{21} & L_{21}^2 + L_{22}^2 \end{pmatrix} \quad\Longrightarrow\quad L_{11} = \sqrt{\Sigma_{11}},\ L_{21} = \frac{\Sigma_{21}}{L_{11}},\ L_{22} = \sqrt{\Sigma_{22} - L_{21}^2}.$$

Read column by column, the same matching gives the general algorithm below. The square roots exist exactly when Σ is positive definite, so a failing Cholesky factorisation is the standard numerical test that a matrix is not a valid covariance.

The standard normals themselves are produced by libraries from uniform random numbers, for example by the Box–Muller transform [2], the polar-coordinate computation of Section 4.2 run backwards: for independent uniforms $u_1 \in (0, 1]$ and $u_2 \in [0, 1)$, the numbers $z_1 = \sqrt{-2\log u_1}\cos(2\pi u_2)$ and $z_2 = \sqrt{-2\log u_1}\sin(2\pi u_2)$ are two independent standard normals, at $O(1)$ cost per pair.

Worked example — drawing one sample by hand

For $\boldsymbol{\mu} = (1, 2)$ and $\Sigma = \left(\begin{smallmatrix} 5 & 2 \\ 2 & 2 \end{smallmatrix}\right)$ of Section 4.1: $L_{11} = \sqrt{5} \approx 2.2361$, $L_{21} = 2/\sqrt{5} \approx 0.8944$, $L_{22} = \sqrt{2 - 4/5} = \sqrt{1.2} \approx 1.0954$. Check: $L_{21}^2 + L_{22}^2 = 0.8 + 1.2 = 2$. With the standard-normal draw $\mathbf{z} = (1, -0.5)$,

$$\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 2.2361 & 0 \\ 0.8944 & 1.0954 \end{pmatrix}\begin{pmatrix} 1 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 1 + 2.2361 \\ 2 + 0.8944 - 0.5477 \end{pmatrix} = \begin{pmatrix} 3.2361 \\ 2.3467 \end{pmatrix}.$$

The first coordinate uses only z_1; the second mixes in z_1 through L_{21}, which is how the correlation enters.

Running time — sampling a multivariate Gaussian

Factorisation: Cholesky costs about $d^3/3$ floating-point operations ($d^3/6$ multiply–adds), $O(d^3)$ — the double loop over (i, j) with an inner sum of length j — and is done once; memory $O(d^2)$ for L. *Per sample*: $O(d)$ for the normals plus $O(d^2)$ for the triangular product $L\mathbf{z}$. *Total* for s samples: $O(d^3 + s\,d^2)$; stacking the $\mathbf{z}$'s as columns of a $d \times s$ matrix turns the loop into one matrix product. For $d = 10^3$ the factorisation takes about 3×10^8 operations; for $d \sim 10^5$ a full Σ (10^{10} entries, 80 GB in double precision) exceeds a workstation or a GPU and its factorisation costs about 3×10^{14} operations, which is why the isotropic case $\mathbf{x} = \boldsymbol{\mu} + \sigma\mathbf{z}$, at $O(d)$ per sample, dominates in high dimension.

Algorithm 3 Cholesky factorisation and sampling from $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$

Require: symmetric positive-definite $\Sigma \in \mathbb{R}^{d\times d}$, mean $\boldsymbol{\mu} \in \mathbb{R}^d$, number of samples s
Ensure: lower-triangular L with $LL^{\mathsf{T}} = \Sigma$; samples $\mathbf{x}_1, \dots, \mathbf{x}_s$

function CHOLESKY(Σ)
  for $j = 1, \dots, d$ **do**
    $a \leftarrow \Sigma_{jj} - \sum_{k<j} L_{jk}^2$
    if $a \le 0$ **then stop**: Σ is not positive definite
    end if
    $L_{jj} \leftarrow \sqrt{a}$
    for $i = j+1, \dots, d$ **do**
      $L_{ij} \leftarrow \big(\Sigma_{ij} - \sum_{k<j} L_{ik}L_{jk}\big)/L_{jj}$
    end for
  end for
  return L ▷ entries above the diagonal are zero
end function
function SAMPLEGAUSSIAN($\boldsymbol{\mu}, \Sigma, s$)
  $L \leftarrow$ CHOLESKY(Σ) ▷ once, reused for every sample
  for $t = 1, \dots, s$ **do**
    draw $\mathbf{z} \in \mathbb{R}^d$ with independent $\mathcal{N}(0,1)$ entries
    $\mathbf{x}_t \leftarrow \boldsymbol{\mu} + L\mathbf{z}$ ▷ equation (10)
  end for
  return $\mathbf{x}_1, \dots, \mathbf{x}_s$
end function

Modern connection — Gaussian noise drives generative models

Equation (10) separates the randomness ($\mathbf{z}$) from the parameters ($\boldsymbol{\mu}, L$). Variational autoencoders exploit this "reparameterisation trick" to differentiate through sampling [5]. Diffusion models, behind many of today's image generators, go further. Their *forward process* gradually turns data into noise: with $\bar{\alpha}_t$ decreasing from 1 to nearly 0,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, I),$$

so $\mathbf{x}_t \mid \mathbf{x}_0 \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)I)$, an isotropic Gaussian (Figure 5). The two coefficients are chosen so that the variance is preserved. Because $\mathbf{z}$ is drawn independently of $\mathbf{x}_0$, the variances of the two terms add: if each component of $\mathbf{x}_0$ has unit variance, then $\mathrm{Var}(x_{t,j}) = \bar{\alpha}_t \cdot 1 + (1-\bar{\alpha}_t)\cdot 1 = 1$ for every component j at every step. A neural network is trained to predict $\mathbf{z}$ from $\mathbf{x}_t$ and t; running the process backwards from pure noise then generates new samples [4, 9]. Everything in this section — isotropic covariances, scaling and shifting, sampling by (10) — is used literally.

6 Learning the parameters

Given $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, the maximum-likelihood estimates of a Gaussian are the empirical mean and covariance,

$$\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i, \qquad \hat{\Sigma} = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^{\mathsf{T}} = \frac{1}{n}X_c^{\mathsf{T}}X_c, \tag{11}$$

where X_c is the design matrix of Note 1 with $\hat{\boldsymbol{\mu}}$ subtracted from every row. Entry by entry, $\hat{\Sigma}_{jk} = \frac{1}{n}\sum_i (x_{i,j} - \hat{\mu}_j)(x_{i,k} - \hat{\mu}_k)$.

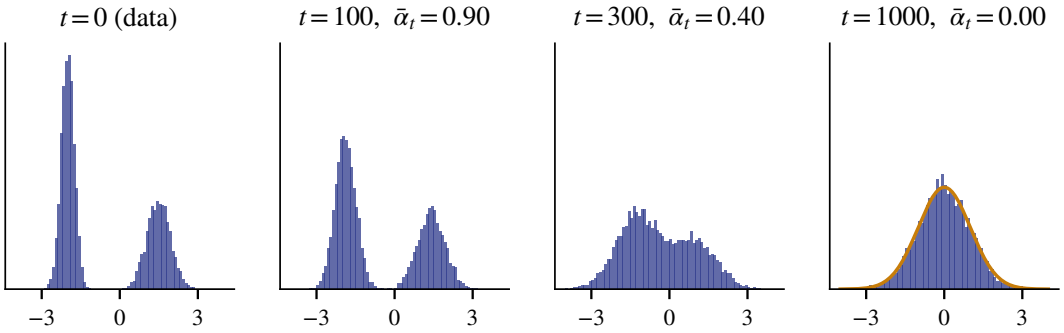


Figure 5. The forward process of a diffusion model applied to a one-dimensional, two-peaked data distribution (histograms of 10^4 samples, the noise schedule of Ho *et al.* [4] with 1000 steps). By the last step the data are indistinguishable from $\mathcal{N}(0, 1)$ (curve).

6.1 Maximum likelihood in one dimension

Write $v = \sigma^2$ to avoid confusing derivatives with respect to σ and σ^2. Taking the logarithm of (5) for each point and adding,

$$\ell(\mu, v) = \sum_{i=1}^{n} \log \mathcal{N}(x_i; \mu, v) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log v - \frac{1}{2v}\sum_{i=1}^{n}(x_i - \mu)^2.$$

Mean. Only the last term depends on μ. Using $\frac{d}{d\mu}(x_i - \mu)^2 = -2(x_i - \mu)$,

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{v}\sum_{i=1}^{n}(x_i - \mu) = \frac{1}{v}\Big(\sum_i x_i - n\mu\Big) = 0 \quad \Longrightarrow \quad \hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i = \bar{x}.$$

Variance. With $\frac{d}{dv}\log v = 1/v$ and $\frac{d}{dv}(1/v) = -1/v^2$,

$$\frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{1}{2v^2}\sum_{i=1}^{n}(x_i - \mu)^2 = 0 \quad \Longrightarrow \quad \hat{v} = \hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2,$$

after multiplying by $2v^2/n$ and inserting $\mu = \hat{\mu}$. Both stationary points are maxima: ℓ is a downward parabola in μ, and in v it tends to $-\infty$ both as $v \to 0$ and as $v \to \infty$, with a single stationary point in between.

> **Worked example — fitting a one-dimensional Gaussian**
>
> Five repeated measurements give $4, 7, 5, 9, 5$. The mean is $\bar{x} = 30/5 = 6$. The deviations are $-2, 1, -1, 3, -1$ (they sum to zero, a useful check), their squares $4, 1, 1, 9, 1$ sum to 16, so $\hat{\sigma}^2 = 16/5 = 3.2$ and $\hat{\sigma} \approx 1.79$. The unbiased version of the next subsection divides by $n - 1 = 4$ and gives 4.0.

6.2 The $1/n$ variance is biased

An estimator is *unbiased* if its average over many datasets equals the true value. Let $x_1, \dots, x_n$ be i.i.d. with mean μ and variance σ^2; in this subsection the x_i and $\bar{x}$ denote random variables (the data of a dataset not yet drawn). First expand the sum of squared deviations:

$$\sum_i (x_i - \bar{x})^2 = \sum_i x_i^2 - 2\bar{x}\sum_i x_i + n\bar{x}^2 = \sum_i x_i^2 - n\bar{x}^2,$$

using $\sum_i x_i = n\bar{x}$. Next, $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$ (Section 1.2) gives $\mathbb{E}[x_i^2] = \mathrm{Var}(x_i) + \mu^2 = \sigma^2 + \mu^2$. For $\bar{x}$, the vector $(x_1, \dots, x_n)$ has covariance $\sigma^2 I$ because the x_i are independent, and (1) with the row $A = (\frac{1}{n}, \dots, \frac{1}{n})$ gives $\mathrm{Var}(\bar{x}) = \sigma^2 \cdot n/n^2 = \sigma^2/n$, so $\mathbb{E}[\bar{x}^2] = \sigma^2/n + \mu^2$. Therefore

$$\mathbb{E}\Big[\sum_i (x_i - \bar{x})^2\Big] = n(\sigma^2 + \mu^2) - n\Big(\frac{\sigma^2}{n} + \mu^2\Big) = (n-1)\,\sigma^2,$$

and $\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2$. The deviations are measured from $\bar{x}$, which was fitted to the same points and is therefore closer to them than the true μ; one "degree of freedom" is used up by the mean. This is why statistics software often divides by $n-1$. The same holds for every entry of $\hat{\Sigma}$. For the sample sizes of machine learning the difference is negligible (a relative change of 10^{-6} for $n = 10^6$), but for $n = 5$ it is 20%.

6.3 The multivariate case

Before deriving (11) in d dimensions, here it is at work on a small dataset.

Worked example — fitting a mean and a covariance to five points

Fit a two-dimensional Gaussian to $(1,2)$, $(2,3)$, $(3,3)$, $(4,5)$, $(5,7)$.

- *Mean:* $\hat{\boldsymbol{\mu}} = \big(\frac{1+2+3+4+5}{5}, \frac{2+3+3+5+7}{5}\big) = (3,4)$.
- *Centred points* (rows of X_c): $(-2,-2)$, $(-1,-1)$, $(0,-1)$, $(1,1)$, $(2,3)$; each column sums to zero.
- *Sums of products:* $\sum c_1^2 = 4+1+0+1+4 = 10$; $\sum c_2^2 = 4+1+1+1+9 = 16$; $\sum c_1 c_2 = 4+1+0+1+6 = 12$.
- *Covariance:* dividing by $n = 5$, $\hat{\Sigma} = \begin{pmatrix} 2 & 2.4 \\ 2.4 & 3.2 \end{pmatrix}$; dividing by $n-1 = 4$ instead gives $\begin{pmatrix} 2.5 & 3 \\ 3 & 4 \end{pmatrix}$.

The fitted correlation is $2.4/\sqrt{2 \times 3.2} \approx 0.95$: the points lie close to a rising line. The eigenvalues of $\hat{\Sigma}$ are about 5.07 and 0.13, so the fitted ellipse is long and thin, and $|\hat{\Sigma}| = 2 \times 3.2 - 2.4^2 = 0.64$.

The log-likelihood of n points under (6) is

$$\ell(\boldsymbol{\mu}, \Sigma) = -\frac{nd}{2}\log(2\pi) - \frac{n}{2}\log|\Sigma| - \frac{1}{2}\sum_{i=1}^{n}(\mathbf{x}_i - \boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}(\mathbf{x}_i - \boldsymbol{\mu}).$$

We sketch the two steps; Bishop [1], Section 2.3.4, derives the mean and states the covariance result, whose derivation is his Exercise 2.34.

Mean. For a symmetric matrix A, the gradient of the quadratic form $(\mathbf{x} - \boldsymbol{\mu})^{\mathsf{T}} A (\mathbf{x} - \boldsymbol{\mu})$ with respect to $\boldsymbol{\mu}$ is $-2A(\mathbf{x} - \boldsymbol{\mu})$, the vector version of $\frac{d}{d\mu} a(x-\mu)^2 = -2a(x-\mu)$ (Note 7, Section 2). Setting the gradient of ℓ to zero, $\Sigma^{-1}\sum_i(\mathbf{x}_i - \boldsymbol{\mu}) = \mathbf{0}$, and multiplying by Σ gives $\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_i \mathbf{x}_i$. The covariance plays no role, exactly as in one dimension.

Covariance (optional, for the interested reader). The quickest check is a diagonal Σ: then ℓ splits into d one-dimensional log-likelihoods, one per coordinate, and Section 6.1 gives $\hat{\Sigma}_{jj} = \frac{1}{n}\sum_i (x_{i,j} - \hat{\mu}_j)^2$, the diagonal of (11). For a full Σ it is easiest to differentiate with respect to the *precision matrix* $Q = \Sigma^{-1}$, using matrix calculus beyond this course. Since $\log|\Sigma| = -\log|Q|$ and a number equals its own trace,

$\mathbf{a}^{\mathsf{T}}Q\mathbf{a} = \operatorname{tr}(Q\mathbf{a}\mathbf{a}^{\mathsf{T}})$, the log-likelihood at $\boldsymbol{\mu} = \hat{\boldsymbol{\mu}}$ becomes

$$\ell = \text{const} + \frac{n}{2}\log|Q| - \frac{1}{2}\operatorname{tr}(QS), \qquad S = \sum_{i=1}^{n}(\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^{\mathsf{T}}.$$

Two standard matrix derivatives, $\partial \log|Q|/\partial Q = Q^{-1}$ and $\partial \operatorname{tr}(QS)/\partial Q = S$ (treating the d^2 entries of Q as independent; imposing the symmetry of Q changes both gradients in the same way and gives the same stationary point), give

$$\frac{\partial \ell}{\partial Q} = \frac{n}{2}Q^{-1} - \frac{1}{2}S = 0 \quad \Longrightarrow \quad \hat{\Sigma} = Q^{-1} = \frac{S}{n},$$

which is (11). For $d = 1$, $Q = 1/v$ and the two derivatives are $1/Q = v$ and S: the computation is the one of Section 6.1.

6.4 When n is not much larger than d

The matrix $\hat{\Sigma} = X_c^{\mathsf{T}}X_c/n$ has rank at most $n-1$ (the n rows of X_c sum to zero, so at most $n-1$ of them are independent), so for $n \le d$ it is singular: the fitted Gaussian is flat in some directions and (6) cannot be evaluated. Even for n somewhat larger than d its small eigenvalues are badly underestimated, and directions that happened to show little spread in the sample are treated as if almost nothing could vary along them. The usual remedies are to restrict the family (diagonal or isotropic) or to *shrink*,

$$\hat{\Sigma}_\epsilon = \hat{\Sigma} + \epsilon I \qquad \text{or} \qquad \hat{\Sigma}_\gamma = (1-\gamma)\,\hat{\Sigma} + \gamma\,\frac{\operatorname{tr}\hat{\Sigma}}{d}\,I, \quad 0 \le \gamma \le 1.$$

Adding ϵI keeps the eigenvectors and raises every eigenvalue by ϵ, so the matrix becomes invertible; the second form pulls the eigenvalues towards their average, and Ledoit and Wolf [6] showed how to choose γ from the data. Both are the simplest instance of the regularisation discussed in Note 4, Section 7: a little bias in exchange for a large reduction in variance.

Algorithm 4 Fitting a Gaussian by maximum likelihood, with optional shrinkage

Require: data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ (rows of X); family $\in$ {full, diagonal, isotropic}; $\epsilon \ge 0$
Ensure: $\hat{\boldsymbol{\mu}} \in \mathbb{R}^d$ and $\hat{\Sigma} \in \mathbb{R}^{d\times d}$

function FitGaussian(X, family, ϵ)
  $\hat{\boldsymbol{\mu}} \leftarrow \frac{1}{n}\sum_{i=1}^{n}\mathbf{x}_i$ ▷ column means
  $X_c \leftarrow X - \mathbf{1}\hat{\boldsymbol{\mu}}^{\mathsf{T}}$ ▷ subtract the mean from every row
  if family = full **then**
    $\hat{\Sigma} \leftarrow \frac{1}{n}X_c^{\mathsf{T}}X_c$ ▷ equation (11)
  else if family = diagonal **then**
    $\hat{\Sigma} \leftarrow \operatorname{diag}(v_1, \dots, v_d)$ with $v_j = \frac{1}{n}\sum_i (X_c)_{ij}^2$
  else
    $\hat{\Sigma} \leftarrow v\,I$ with $v = \frac{1}{nd}\sum_{i,j}(X_c)_{ij}^2$
  end if
  $\hat{\Sigma} \leftarrow \hat{\Sigma} + \epsilon I$ ▷ shrinkage; $\epsilon = 0$ is plain maximum likelihood
  return $\hat{\boldsymbol{\mu}}, \hat{\Sigma}$
end function

Running time — fitting a Gaussian

Mean and centring: $O(nd)$, one pass over the data. *Full covariance*: $O(nd^2)$ for the product $X_c^{\mathsf{T}}X_c$ (d^2 entries, each a sum of n terms; half of them suffice by symmetry). *Diagonal or isotropic*: $O(nd)$. *Shrinkage*: $O(d)$. Memory: $O(d^2)$ for a full $\hat{\Sigma}$, $O(d)$ otherwise. For $n = 10^6$ and $d = 10^3$ the product $X_c^{\mathsf{T}}X_c$ is 10^{12} multiply–adds, but a single matrix multiplication, which GPUs do best. There is no training loop: the fit is in

closed form, unlike most models of this course (Note 7).

Worked example — two points in two dimensions

From $(0,0)$ and $(2,2)$: $\hat{\boldsymbol{\mu}} = (1,1)$ and $\hat{\Sigma} = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, with eigenvalues 2 (along $(1,1)$) and 0 (along $(1,-1)$) and determinant 0. The fitted Gaussian lives on the line $x_1 = x_2$ and assigns zero density to every point off it. With $\epsilon = 0.1$ the eigenvalues become 2.1 and 0.1, and $|\hat{\Sigma}_\epsilon| = 0.21$. Now every point has a finite Mahalanobis distance. With the eigenvectors $\mathbf{v}_1 = (1,1)/\sqrt{2}$, $\mathbf{v}_2 = (1,-1)/\sqrt{2}$ and $\Delta^2 = (\mathbf{r}^\mathsf{T}\mathbf{v}_1)^2/2.1 + (\mathbf{r}^\mathsf{T}\mathbf{v}_2)^2/0.1$: the training point $(2,2)$ has $\mathbf{r} = (1,1)$, $\mathbf{r}^\mathsf{T}\mathbf{v}_1 = \sqrt{2}$, $\mathbf{r}^\mathsf{T}\mathbf{v}_2 = 0$, so $\Delta^2 = 2/2.1 \approx 0.95$; the point $(1,0)$, which lies off the line, has $\mathbf{r} = (0,-1)$, $\mathbf{r}^\mathsf{T}\mathbf{v}_1 = -1/\sqrt{2}$, $\mathbf{r}^\mathsf{T}\mathbf{v}_2 = 1/\sqrt{2}$, so $\Delta^2 = 0.5/2.1 + 0.5/0.1 \approx 0.24 + 5 = 5.24$. The $1/\epsilon$ term dominates. The value of ϵ sets how strongly "off the line" is penalised; it is chosen on validation data (Note 4).

6.5 Computing log-densities

In practice one evaluates $\log \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma)$, never the density itself, which underflows in high dimension: with $d = 1000$, $|\Sigma| = 1$ and $\Delta^2 \approx d$, the factor $e^{-\Delta^2/2} \approx e^{-500} \approx 10^{-217}$ and the prefactor $(2\pi)^{-d/2} \approx 10^{-399}$ together give a density of order 10^{-616}, far below the smallest positive double-precision number (about 10^{-308}, or 10^{-324} counting subnormals), while its logarithm, about -1419, is an ordinary number. With $\Sigma = LL^\mathsf{T}$, solve the triangular system $L\mathbf{u} = \mathbf{x} - \boldsymbol{\mu}$; then

$$\Delta^2 = (\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}(LL^\mathsf{T})^{-1}(\mathbf{x}-\boldsymbol{\mu}) = (L^{-1}(\mathbf{x}-\boldsymbol{\mu}))^\mathsf{T}(L^{-1}(\mathbf{x}-\boldsymbol{\mu})) = \|\mathbf{u}\|^2, \qquad \log|\Sigma| = \log|L|^2 = 2\sum_i \log L_{ii},$$

because the determinant of a triangular matrix is the product of its diagonal. No matrix is ever inverted: a triangular system is solved by *forward substitution*, $u_1 = r_1/L_{11}$, then $u_2 = (r_2 - L_{21}u_1)/L_{22}$, and so on, which is both faster and more accurate than forming Σ^{-1}. In numpy the two steps are `np.linalg.cholesky` and `scipy.linalg.solve_triangular`, the second applied to all rows of X at once (Algorithm 5).

Algorithm 5 Gaussian log-density of many points via Cholesky

Require: points $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$; mean $\boldsymbol{\mu}$; positive-definite Σ
Ensure: $g_i = \log \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}, \Sigma)$ for $i = 1, \dots, n$

function GaussLogPdf($X, \boldsymbol{\mu}, \Sigma$)
  $L \leftarrow$ Cholesky(Σ) ▷ Algorithm 3, once
  $h \leftarrow \sum_{j=1}^{d} \log L_{jj}$ ▷ $h = \frac{1}{2}\log|\Sigma|$
  for $i = 1, \dots, n$ **do**
    $\mathbf{r} \leftarrow \mathbf{x}_i - \boldsymbol{\mu}$
    for $j = 1, \dots, d$ **do** ▷ forward substitution for $L\mathbf{u} = \mathbf{r}$
      $u_j \leftarrow \left(r_j - \sum_{k<j} L_{jk}u_k\right)/L_{jj}$
    end for
    $g_i \leftarrow -\frac{1}{2}\|\mathbf{u}\|^2 - h - \frac{d}{2}\log(2\pi)$
  end for
  return $g_1, \dots, g_n$
end function

Running time — evaluating Gaussian log-densities

Set-up: Cholesky $O(d^3)$ and the log-determinant $O(d)$, once per Gaussian. *Per query point*: $O(d^2)$ for the forward substitution (row j needs j products) plus $O(d)$ for the norm. *Total*: $O(d^3 + nd^2)$ time and $O(d^2)$ memory, plus $O(nd)$ if all points are solved at once. Inverting Σ explicitly costs the same order but loses

accuracy when Σ is badly conditioned. With a diagonal Σ everything is $O(d)$ per point. In a Gaussian Bayes classifier (Note 6, Section 3) the set-up is repeated for each of the m classes: $O(md^3 + nmd^2)$.

Worked example — a log-density by Cholesky

For the example of Section 4.1 and the point $A = (3, 3)$, with the factor L of Section 5,

$$\mathbf{r} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad L = \begin{pmatrix} 2.236 & 0 \\ 0.894 & 1.095 \end{pmatrix}.$$

Forward substitution gives $u_1 = 2/2.236 = 0.894$ and $u_2 = (1 - 0.894 \times 0.894)/1.095 = 0.183$, so $\|\mathbf{u}\|^2 = 0.800 + 0.033 = 0.833 = 5/6$, the value found before with Σ^{-1}. With $h = \log 2.236 + \log 1.095 = 0.805 + 0.091 = 0.896 = \frac{1}{2}\log 6$, the log-density is $-\frac{1}{2} \cdot \frac{5}{6} - h - \frac{d}{2}\log(2\pi)$ with $d = 2$, i.e. $-0.417 - 0.896 - 1.838 \approx -3.150$ (exactly -3.1504), and $e^{-3.150} \approx 0.0428$ as in Section 4.1.

Modern connection — Mahalanobis distance as an anomaly detector

A trained classifier can be fooled by inputs unlike anything it has seen. A simple and strong defence fits a Gaussian to the network's internal feature vectors for each class (with a shared covariance) and flags a new input whose features have a large Mahalanobis distance to every class mean [7]. The same recipe is used to monitor detectors and industrial processes. If the Gaussian model and its parameters were exact, Δ^2 would follow a χ^2 distribution with d degrees of freedom (Section 4.2). In $d = 2$, $P(\Delta > k) = e^{-k^2/2} = 0.05$ gives $k^2 = -2\ln 0.05 = 5.99$, so an alarm at $\Delta^2 > 5.99$ would fire for 5% of normal events; with fitted parameters the threshold is tuned on held-out normal data instead.

7 The algorithms of this note at a glance

Table 1 collects the running times derived above. All fits of this note are in *closed form* (counting, averaging, one matrix product), with no iterative training; their price is paid in d, since fitting a full covariance costs $O(d^2)$ per point, and sampling from it, evaluating it or conditioning it adds an $O(d^3)$ factorisation, paid once.

Table 1. Costs of the algorithms of this note: n data points, d dimensions, m categories, s samples drawn.

Algorithm	Set-up / training	Per sample or query	Memory
Empirical c.d.f. (sort, binary search)	$O(n \log n)$	$O(\log n)$	$O(n)$
Fit categorical (Alg. 1)	$O(n + m)$	—	$O(m)$
Sample categorical, linear scan	—	$O(m)$	$O(1)$
Sample categorical, binary search	$O(m)$	$O(\log m)$	$O(m)$
Cholesky factorisation	$O(d^3)$	—	$O(d^2)$
Condition on $\mathbf{x}_b$ (Alg. 2)	$O(d_b^3 + d_a d_b^2 + d_a^2 d_b)$	$O(d_b^2 + d_a d_b)$	$O(d^2)$
Sample $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ (Alg. 3)	$O(d^3)$	$O(d^2)$	$O(d^2)$
Fit Gaussian, full (Alg. 4)	$O(nd^2)$	—	$O(d^2)$
Fit Gaussian, diagonal or isotropic	$O(nd)$	—	$O(d)$
Log-density, full (Alg. 5)	$O(d^3)$	$O(d^2)$	$O(d^2)$
Log-density, diagonal	$O(d)$	$O(d)$	$O(d)$

Summary A distribution is given by a c.d.f., a probability table or a density; marginals, conditionals and Bayes' rule relate its parts, and $\mathrm{Cov}(A\mathbf{X}+\mathbf{b}) = A\Sigma A^{\mathsf{T}}$ tells how spreads transform. We sample, evaluate, infer and learn; maximum likelihood gives the observed frequencies for a categorical (pseudo-counts cure zero counts), the cross-entropy for language models and least squares for Gaussian noise. The multivariate Gaussian has elliptical contours with axes along the eigenvectors of Σ; the Mahalanobis distance measures distance in units of that spread. It is closed under marginalising and conditioning, sampled as $\boldsymbol{\mu} + L\mathbf{z}$, and fitted by the empirical mean and covariance, at a cost of $O(nd^2)$, with shrinkage when d is comparable to n.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. A colleague computes a probability density and obtains the value 4 at some point. Is this an error? Explain the difference between a density and a probability, and state the units of $p(x)$ when x is a length measured in metres.

A2. In the particle-identification example of Section 1.1 the tagger finds 90% of kaons, yet only two thirds of tagged tracks are kaons. Explain in words why. Without a detailed calculation, would the fraction of kaons among tagged tracks go up or down if kaons made up 50% of the tracks? Why?

A3. Why do we maximise the log-likelihood rather than the likelihood itself? Give one mathematical and one numerical reason, and explain why the maximiser is the same.

A4. A categorical model is fitted by counting on a small dataset in which one category never occurs. What probability does maximum likelihood give it, and what happens to the test log-likelihood if that category appears in the test set? How does a pseudo-count fix this, and why does its effect disappear as n grows?

A5. A language model samples its next token with temperature T. Describe what happens for $T \to 0$, for $T = 1$ and for very large T, and why one might choose $T < 1$ for answering factual questions.

A6. True or false, with a justification: "Fitting a regression function by least squares is maximum likelihood under Gaussian noise only if the noise variance σ^2 is known."

A7. True or false, with a justification: (a) independent variables are uncorrelated; (b) uncorrelated variables are independent; (c) uncorrelated *jointly Gaussian* variables are independent.

A8. Two points lie at the same Euclidean distance from the mean of a correlated two-dimensional Gaussian, yet one has Mahalanobis distance 0.9 and the other 2.2. Explain with a sketch of the contour ellipse which one lies along the long axis, and which one is the more unusual observation.

A9. You must model data with $d = 100$ features from $n = 150$ examples. Compare the isotropic, diagonal and full covariance families by their number of parameters and say which you would

try first, and why.

A10. In the conditional Gaussian (9), the conditional variance does not depend on the observed value $\mathbf{x}_b$. Interpret this statement. In two dimensions, use $\mathrm{Var}(X_1 \mid X_2) = \Sigma_{11}(1-\rho^2)$ to say whether observing X_2 can ever increase the variance of X_1.

A11. Why does the "68% within one standard deviation" rule fail for the Mahalanobis distance in $d = 2$? Is the probability inside the ellipse $\Delta \le 1$ larger or smaller than 68%, and what happens to it as d grows?

A12. To sample $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ we need a matrix L with $LL^{\mathsf{T}} = \Sigma$. Would $\mathbf{x} = \boldsymbol{\mu} + \Sigma\mathbf{z}$ also work? Is the Cholesky factor the only valid choice of L?

A13. Why does the maximum-likelihood variance divide by n while many textbooks divide by $n-1$? Does the choice matter for $n = 5$? For $n = 10^6$?

A14. Why is the empirical covariance matrix singular when $n \le d$? What does adding ϵI do to its eigenvalues and eigenvectors, and what is the price of this "fix"?

A15. Give two reasons why measurement noise is often modelled as Gaussian. Describe a situation in which the central-limit argument fails.

A16. A language model samples from $m = 10^5$ tokens. Compare the cost per draw of a linear scan and of binary search over the c.d.f., and explain why neither is the bottleneck when the probabilities change at every token.

A17. Explain why evaluating a full-covariance Gaussian at n points costs $O(d^3)$ once plus $O(d^2)$ per point, what changes for a diagonal covariance, and why one solves triangular systems with the Cholesky factor instead of forming Σ^{-1}.

B. Short calculations

B1. A detector classifies events into four categories with counts $5, 10, 20, 15$. (a) Give the maximum-likelihood probabilities. (b) Give the estimates with a pseudo-count $\alpha = 1$. (c) Number the categories 1–4 and, using the probabilities of (a), write down the c.d.f. and find the category returned by inverse-c.d.f. sampling for $u = 0.62$. (d) How many comparisons does binary search need in the worst case for $m = 1024$ categories?

B2. A resistor value is $X \sim \mathcal{N}(100, 4^2)$ ohm. Using $\Phi(1) = 0.841$, $\Phi(2) = 0.977$ and $\Phi(1.96) = 0.975$, compute $P(X > 108)$, $P(92 < X < 104)$ and the symmetric interval around 100 that contains 95% of the resistors.

B3. For $\boldsymbol{\mu} = \mathbf{0}$ and $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ with $|\rho| < 1$, find the eigenvalues and eigenvectors of Σ and use (9) to compute $p(x_1 \mid x_2)$. For $\rho = 0.9$, what is the conditional variance of x_1, and what fraction of its original variance is removed by observing x_2?

B4. For $\boldsymbol{\mu} = \mathbf{0}$ and $\Sigma = \mathrm{diag}(4, 1)$, compute the Euclidean and the Mahalanobis distances of $(2, 0)$ and $(0, 2)$ from the mean. Which point is more typical? Give the half-axes of the ellipse $\Delta = 1$.

B5. Find the Cholesky factor of $\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix}$, check that $LL^{\mathsf{T}} = \Sigma$, and transform the standard-normal draw $\mathbf{z} = (1, -1)$ into a sample of $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ with $\boldsymbol{\mu} = (1, 0)$.

B6. Fit a two-dimensional Gaussian to the four points $(0,1)$, $(2,1)$, $(2,3)$, $(4,3)$: give $\hat{\boldsymbol{\mu}}$, $\hat{\Sigma}$ (dividing by n), the version dividing by $n-1$, and the correlation coefficient.

B7. Measured lifetimes 0.5, 1.0 and 2.5 μs are modelled by the exponential density $p(x) = \lambda e^{-\lambda x}$ for $x \geq 0$. Write the log-likelihood, set its derivative with respect to λ to zero, and compute $\hat{\lambda}$.

C. Derivations

C1. Show that the maximum-likelihood estimate of a categorical distribution from counts $n_1, \dots, n_m$ is $\hat{\pi}_k = n_k / n$, by maximising $\sum_k n_k \log \pi_k$ subject to $\sum_k \pi_k = 1$ with a Lagrange multiplier. What does it predict for a category never observed, and why is this a problem for a language model? Show that the pseudo-count estimate (4) is the maximum-likelihood estimate for the counts $n_k + \alpha$.

C2. Verify that (5) integrates to one and has mean μ and variance σ^2. (Use $\int_{-\infty}^{\infty} e^{-z^2/2}\,dz = \sqrt{2\pi}$, obtained by squaring the integral and changing to polar coordinates as in Section 4.2.)

C3. If the noise has the Laplace density $p(\varepsilon) = \frac{1}{2b} e^{-|\varepsilon|/b}$, which loss does maximum likelihood minimise? Which estimate of a constant θ minimises $\sum_i |y_i - \theta|$, and why is it less sensitive to outliers than the mean?

C4. Let $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_d)$. Show that $\mathbb{E}\,\|\mathbf{z}\|^2 = d$ and $\operatorname{Var}\|\mathbf{z}\|^2 = 2d$ (use $\mathbb{E}[z^4] = 3$ for a standard normal). Conclude that for large d almost all samples lie near a sphere of radius $\sqrt{d}$, far from the mode at the origin. (Note 3 returns to this.)

C5. Prove the general conditional formula (9): define $\mathbf{e} = \mathbf{x}_a - \boldsymbol{\mu}_a - B(\mathbf{x}_b - \boldsymbol{\mu}_b)$ with $B = \Sigma_{ab}\Sigma_{bb}^{-1}$, show with (1) that $\operatorname{Cov}(\mathbf{e}, \mathbf{x}_b) = 0$, and compute $\operatorname{Cov}(\mathbf{e})$.

D. Programming

D1. Implement SampleLinear and SampleBinary of Algorithm 1 with numpy. Check that the frequencies of 10^5 samples match π, and time both samplers for $m = 10, 10^3, 10^5$ with probabilities drawn at random. Where does the $O(\log m)$ advantage become visible?

D2. Implement sampling by (10) and Algorithm 5 (vectorised over the points). Draw n samples from a Gaussian in $d = 50$ with a random full covariance, refit by (11), and plot the average test log-likelihood against n for the full, diagonal and shrunk $(\hat{\Sigma} + \epsilon I)$ estimates. Which wins for small n?

D3. Implement Cholesky of Algorithm 3 with explicit loops and compare it with the library routine `np.linalg.cholesky` for accuracy and running time on random positive-definite matrices with $d = 50, 200, 800$. Does the time grow like d^3? Then sample 10^5 points from the two-dimensional example of Section 4.1, and check that about 39%, 86% and 99% fall inside $\Delta = 1, 2, 3$.

Further reading. Bishop [1], sections 2.1–2.3; Murphy [8], chapters 2–3; for data analysis in physics, Cowan [3].

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[2] G. E. P. Box and M. E. Muller, A note on the generation of random normal deviates, *Ann. Math. Statist.* 29(2), 610–611 (1958).

[3] G. Cowan, *Statistical Data Analysis*, Oxford University Press (1998).

[4] J. Ho, A. Jain and P. Abbeel, Denoising diffusion probabilistic models, *NeurIPS* (2020).

[5] D. P. Kingma and M. Welling, Auto-encoding variational Bayes, *ICLR* (2014).

[6] O. Ledoit and M. Wolf, A well-conditioned estimator for large-dimensional covariance matrices, *J. Multivariate Anal.* 88(2), 365–411 (2004).

[7] K. Lee, K. Lee, H. Lee and J. Shin, A simple unified framework for detecting out-of-distribution samples and adversarial attacks, *NeurIPS* (2018).

[8] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[9] Y. Song *et al.*, Score-based generative modeling through stochastic differential equations, *ICLR* (2021).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.